

# МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ВЫПОЛНЕНИЮ ЛАБОРАТОРНОЙ РАБОТЫ

**Направление подготовки:** 09.04.02 «Информационные системы и технологии»

**Уровень образования:** Магистратура

**Дисциплина:** «Параллельное программирование»

---

## 1. Тема лабораторной работы

Разработка и профилирование модели распределённой вычислительной системы для ресурсоёмких задач (математическое моделирование, обработка спутниковых изображений и пр.): анализ архитектуры вычислительного узла, построение математической модели вычислительной системы, измерение пропускной способности памяти и оптимизация алгоритма скалярного произведения векторов.

---

## 2. Цель и задачи лабораторной работы

**Цель работы:** Формирование у магистрантов компетенций в области проектирования, моделирования и профилирования высокопроизводительных вычислительных систем (HPC), а также приобретение практических навыков низкоуровневой оптимизации вычислительных алгоритмов, критически важных для обработки больших массивов данных дистанционного зондирования Земли (ДЗЗ).

**Задачи лабораторной работы:** 1. Изучить технические характеристики современных вычислительных узлов (серверные материнские платы, архитектура CPU, иерархия кэш-памяти, контроллеры памяти, шина PCI-Express, чипсеты) и сетевые технологии HPC (InfiniBand, RoCE). 2. Освоить методику замера пропускной способности подсистемы памяти (Memory Bandwidth) и оценить её влияние на производительность операций чтения/записи больших растровых данных. 3. Реализовать алгоритм вычисления скалярного произведения векторов на языках Python и C++, исследовать влияние флагов оптимизации компилятора g++ на время выполнения и сгенерированный машинный код. 4. Провести статический анализ ассемблерного кода с использованием инструмента Compiler Explorer (godbolt.org) для выявления применённых компилятором оптимизаций (векторизация SIMD, развёртка циклов, FMA). 5. Сформулировать выводы о целесообразности применения тех или иных архитектурных и программных решений при разработке распределённых систем обработки спутниковых изображений.

---

## 3. Программное и аппаратное обеспечение

**Аппаратное обеспечение:** \* Вычислительный узел на базе архитектуры x86-64 (Intel Core i7/i9/Xeon или AMD Ryzen/EPYC). \* Оперативная память DDR4/DDR5 (двух- или

четырёхканальный режим). \* Накопитель NVMe SSD (для хранения эталонных наборов данных).

**Программное обеспечение:** \* Операционная система: Linux (Ubuntu 20.04/22.04 LTS) или Windows 10/11 с подсистемой WSL2. \* Компиляторы: GCC 9+ или Clang 10+ (с поддержкой стандартов C++17/C++20). \* Языки программирования: Python 3.8+ (с библиотеками `numpy`, `timeit`), C++. \* Системы сборки: CMake 3.16+. \* Инструменты профилирования и анализа: `perf`, `lscpu`, `hwloc`, `likwid` (опционально), онлайн-компилятор Compiler Explorer ([godbolt.org](https://godbolt.org)).

---

## 4. Введение

### 4.1. Актуальность темы

Современные системы дистанционного зондирования Земли (ДЗЗ), такие как спутниковые группировки Sentinel (программа Copernicus), Landsat 8/9 и коммерческие аппараты высокого разрешения (WorldView, Pleiades), генерируют колоссальные объёмы данных. Один снимок Sentinel-2 в формате Level-1C занимает около 1 ГБ, а глобальная мозаика поверхности суши обновляется каждые 5 дней. Обработка таких массивов данных, включающая атмосферную коррекцию, панорамную склейку, классификацию земного покрова и мониторинг чрезвычайных ситуаций, требует экстремальной вычислительной эффективности.

В высоконагруженных распределённых вычислительных системах (HPC) «узким местом» (bottleneck) зачастую является не пиковая вычислительная мощность процессора (измеряемая в FLOPS), а пропускная способность подсистемы памяти (Memory Bandwidth) и задержки при обращении к ней (Memory Latency). Эта фундаментальная проблема, известная как «Стена памяти» (Memory Wall), была впервые сформулирована В.А. Вульфом и С.А. Макки в 1995 году и остаётся актуальной. Операции, требующие частого обращения к оперативной памяти (memory-bound operations), такие как поэлементное преобразование цветовых пространств, наложение фильтров или вычисление спектральных индексов, деградируют в производительности из-за промахов кэша (cache misses).

Понимание микроархитектуры вычислительного узла (иерархия кэшей, NUMA-архитектура, ширина шины памяти) и владение методами низкоуровневой оптимизации (векторизация SIMD, управление выравниванием данных, выбор оптимальных флагов компиляции) является критически важным навыком для инженера, разрабатывающего распределённые системы обработки спутниковых изображений. Неоптимизированный код может обрабатывать один кадр несколько секунд, тогда как оптимизированный код с использованием аппаратных возможностей способен обрабатывать потоки данных в реальном времени.

### 4.2. Обзор научных работ

Проблема оптимизации алгоритмов обработки изображений и сигналов в распределённых системах широко освещена в рецензируемой научной литературе высшего уровня.

В фундаментальной работе *F. Q. da Silva et al. (IEEE Transactions on Geoscience and Remote Sensing, 2020)* «Memory-efficient processing of remote sensing images» авторы демонстрируют, что при обработке гиперспектральных и мультиспектральных данных до 80% времени выполнения тратится на операции чтения/записи в оперативную память, а не на непосредственные арифметические вычисления. Авторы предлагают методы тайлинга (tiling) и оптимизации паттернов доступа к памяти, доказывая, что локализация данных в кэше L1/L2 критична для ускорения алгоритмов классификации.

Исследование *H. Li et al. (ISPRS Journal of Photogrammetry and Remote Sensing, 2021)* «Real-time orthorectification of UAV and satellite imagery using SIMD-optimized algorithms» посвящено применению инструкций SIMD (AVX2/AVX-512) для ускорения геометрических преобразований и вычисления скалярных произведений при спектральном сопоставлении. Показано, что ручная векторизация или использование универсальных интринсиков позволяет достичь ускорения в 4–8 раз по сравнению со скалярным кодом на уровне исходников. Авторы подчеркивают, что автоматическая векторизация компилятором (auto-vectorization) часто оказывается неэффективной из-за сложных паттернов доступа к данным, если не заданы строгие флаги оптимизации (-O3, -march=native, -ffast-math).

Вопросы архитектуры распределённых вычислительных систем рассматриваются в работе *A. P. Druskov et al. (Journal of Parallel and Distributed Computing, 2022)*, где обосновывается необходимость учёта топологии сети (InfiniBand, RoCEv2) и NUMA-архитектуры серверных узлов при распараллеливании задач обработки больших растров. Показано, что игнорирование локальности памяти при выделении буферов в многопроцессорных системах может снизить общую производительность кластера на 30-40%.

Таким образом, данная лабораторная работа направлена на приобретение практических навыков решения фундаментальных проблем производительности, описанных в передовых научных публикациях, и формирование компетенций по созданию высокопроизводительных вычислительных моделей.

---

## 5. Теоретические сведения

### 5.1. Технические характеристики вычислительного узла: примеры и расчёты

Современный серверный вычислительный узел представляет собой сложную иерархическую систему. Рассмотрим конкретный пример конфигурации сервера для обработки ДЗЗ: \* **CPU:** 2 × AMD EPYC 7763 (64 ядра каждый, всего 128 ядер). Каждое ядро имеет L1 (32 КБ данных + 32 КБ инструкций), L2 (512 КБ) и разделяемый L3 кэш (32 МБ на каждый CCD-модуль). \* **RAM:** 512 ГБ DDR4-3200 ECC. Контроллер памяти поддерживает 8 каналов на процессор. \* **Расчёт теоретической пропускной способности памяти (Memory Bandwidth):** Формула:  $BW = \text{Частота} \times \text{Байт на такт} \times \text{Каналы} \times \text{Сокеты}$ . Для данной системы:  $3200 \times 10^6 \text{ транзакций/с} \times 8 \text{ байт} \times 8 \text{ каналов} \times 2 \text{ сокета} \approx 409.6 \text{ ГБ/с}$ . *Пример влияния:* При обработке снимка размером 10 ГБ, теоретический минимум времени только на его чтение в память

составит  $10/409.6 \approx 0.024$  секунды. Однако на практике из-за задержек и неидеального доступа это время будет в 2-3 раза выше. \* **Шина PCI-Express:** Версия 4.0 x16 обеспечивает пропускную способность около 32 ГБ/с, что критично для передачи данных с NVMe накопителей или GPU-акселераторов. \* **Сетевые технологии HPC:** В кластерах используется InfiniBand HDR (200 Гбит/с) или RoCEv2. Они поддерживают RDMA (Remote Direct Memory Access), позволяя узлу А записать данные напрямую в RAM узла Б, минуя CPU и ОС узла Б, что снижает задержку (latency) с микросекунд до наносекунд.

## 5.2. Математическая модель: Скалярное произведение в задачах ДЗЗ

Скалярное произведение двух векторов  $\mathbf{a}$  и  $\mathbf{b}$  размерности  $N$  определяется как:

$$S = \mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^N a_i \cdot b_i$$

**Конкретный пример из предметной области:** Алгоритм классификации «Спектральный угол» (Spectral Angle Mapper, SAM). Пусть у нас есть пиксель спутникового снимка с 3 каналами (NIR, Red, Green):  $\mathbf{v}_{pixel} = [1200, 800, 400]$  и эталонный вектор чистой растительности:  $\mathbf{v}_{ref} = [1000, 700, 300]$ . Скалярное произведение:  $S = (1200 \times 1000) + (800 \times 700) + (400 \times 300) = 1\,200\,000 + 560\,000 + 120\,000 = 1\,880\,000$ . Для изображения размером  $10000 \times 10000$  пикселей необходимо выполнить  $10^8$  таких скалярных произведений.

С точки зрения архитектуры, это классическая **memory-bound** задача. На каждые 2 операции с плавающей запятой (умножение и сложение) требуется прочитать 2 элемента из памяти. Арифметическая интенсивность (Arithmetic Intensity) крайне низка, что делает задачу чувствительной к пропускной способности памяти, а не к частоте процессора.

## 5.3. Компиляция g++ с оптимизацией: примеры трансформации кода

Компилятор GCC предоставляет широкий спектр флагов, кардинально меняющих машинный код: \* -O0: Отключение оптимизаций. Код транслируется «как есть». \* -O2: Стандартная оптимизация. Включает свёртку констант, удаление мёртвого кода. \* -O3: Агрессивная оптимизация. Включает авто-векторизацию (-ftree-vectorize) и развёртку циклов. \* -march=native: Генерирует инструкции, специфичные для текущего CPU (например, AVX2 или AVX-512). \* -ffast-math: Разрешает компилятору нарушать строгие стандарты IEEE 754 (например, менять порядок сложения чисел с плавающей запятой), что даёт значительный прирост скорости, но может незначительно изменить результат из-за погрешностей округления.

**Пример трансформации (Развёртка циклов - Loop Unrolling):** Исходный код:

```
for (int i = 0; i < 4; ++i) sum += a[i] * b[i];
```

После оптимизации -funroll-loops компилятор может сгенерировать:

```
sum += a[0]*b[0] + a[1]*b[1] + a[2]*b[2] + a[3]*b[3];
```

Это устраняет 3 проверки условия цикла и 3 инкремента счётчика, уменьшая нагрузку на конвейер процессора (branch prediction).

**Пример векторизации (SIMD):** Вместо скалярного умножения одного числа, инструкция AVX2 (например, `vmulps`) умножает 8 значений типа `float` одновременно, используя 256-битный регистр `ymm`.

#### 5.4. Способы и библиотеки для замеров производительности: примеры команд

- **Python:** Использование `time.perf_counter()` (монотонные часы с наносекундным разрешением) предпочтительнее `time.time()`, который может быть изменён системной синхронизацией (NTP).
  - **C++:** Использование `std::chrono::high_resolution_clock` или `std::chrono::steady_clock`.
  - **Системный профайлер Linux perf:** Позволяет измерять не только время, но и аппаратные события. *Пример команды:* `perf stat -e cache-misses,cache-references,instructions,cycles,mem_load_retired.l3_hit ./my_program`  
*Пример вывода:* 15.2% cache-misses указывает на то, что 15% обращений к памяти не были найдены в кэше и потребовали доступа к RAM, что является «красным флагом» для оптимизации.
- 

## 6. Программа выполнения лабораторной работы

### Задача 1. Шаблонный проект и замер пропускной способности памяти

**Описание задачи из предметной области:** При обработке спутниковых мозаик (например, при чтении тайлов изображения Sentinel-2 размером 10980×10980 пикселей, тип данных `uint16`) система должна перенести данные из оперативной памяти в кэш процессора для последующей обработки. Скорость этого переноса (пропускная способность памяти) является фундаментальным ограничением производительности всего пайплайна.

**План решения:** 1. Создать шаблонный проект на Python и C++. 2. Сгенерировать массив данных, имитирующий растр спутникового снимка (например, 8000×8000 элементов типа `float32` или `uint16`). 3. Реализовать операцию интенсивного чтения и записи (например, копирование массива или поэлементное масштабирование). 4. Замерить время выполнения и рассчитать фактическую пропускную способность в ГБ/с.

**Пошаговое описание решения задачи:** 1. В Python использовать библиотеку `numpy` для создания массива и `time.perf_counter` для замера времени операции копирования или простого арифметического преобразования. 2. В C++ выделить память с помощью `std::vector` или сырых указателей, инициализировать её, затем выполнить цикл копирования или масштабирования. 3. Рассчитать объём переданных данных:  $V = 2 \times N \times \text{sizeof}(T)$  (чтение + запись). 4. Вычислить пропускную способность:  $BW = V/t$ , где  $t$  – время в секундах.

### Замер пропускной способности памяти

Программный код на Python:

```
import numpy as np
import time

def measure_memory_bandwidth_python():
    # Моделирование спутникового снимка: 8000 x 8000 пикселей, 3 канала, float32
    # Объём данных: 8000 * 8000 * 3 * 4 байта ≈ 768 МБ
    rows, cols, channels = 8000, 8000, 3
    size_bytes = rows * cols * channels * np.dtype('float32').itemsize

    print(f"Создание массива данных ({size_bytes / (1024**3):.2f} ГБ)...")
    src = np.random.rand(rows, cols, channels).astype(np.float32)
    dst = np.empty_like(src)

    # Замер операции копирования (чтение + запись)
    iterations = 10
    print(f"Выполнение {iterations} итераций копирования...")

    start_time = time.perf_counter()
    for _ in range(iterations):
        np.copyto(dst, src)
    end_time = time.perf_counter()

    elapsed_time = end_time - start_time
    # Объём переданных данных за все итерации (чтение + запись)
    total_data_transferred = iterations * size_bytes * 2

    bandwidth_gbps = (total_data_transferred / (1024**3)) / elapsed_time

    print(f"Общее время: {elapsed_time:.4f} сек")
    print(f"Фактическая пропускная способность памяти: {bandwidth_gbps:.2f} ГБ/с")
    ")

if __name__ == "__main__":
    measure_memory_bandwidth_python()
```

Программный код на C++:

```
#include <iostream>
#include <vector>
#include <chrono>
#include <numeric>

void measure_memory_bandwidth_cpp() {
    const size_t rows = 8000;
    const size_t cols = 8000;
    const size_t channels = 3;
    const size_t total_elements = rows * cols * channels;
    const size_t size_bytes = total_elements * sizeof(float);

    std::cout << "Выделение памяти (" << size_bytes / (1024.0 * 1024.0) << " М
```

```

Б)...\\n";
std::vector<float> src(total_elements, 1.5f);
std::vector<float> dst(total_elements, 0.0f);

const int iterations = 10;
std::cout << "Выполнение " << iterations << " итераций копирования...\\n";

auto start_time = std::chrono::high_resolution_clock::now();

for (int iter = 0; iter < iterations; ++iter) {
    // Использование std::copy, который компилятор оптимизирует в тетсру или
    // векторизованный цикл
    std::copy(src.begin(), src.end(), dst.begin());
}

auto end_time = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> elapsed = end_time - start_time;

// Объём переданных данных (чтение + запись) в гигабайтах
double total_data_gb = (iterations * size_bytes * 2.0) / (1024.0 * 1024.0 *
1024.0);
double bandwidth_gbps = total_data_gb / elapsed.count();

std::cout << "Общее время: " << elapsed.count() << " сек\\n";
std::cout << "Фактическая пропускная способность памяти: " << bandwidth_gbps
<< " ГБ/с\\n";
}

int main() {
    measure_memory_bandwidth_cpp();
    return 0;
}

```

**Анализ решения и формулирование выводов:** Сравнить полученное значение пропускной способности с теоретическим максимумом для используемой подсистемы памяти (например, для двухканальной DDR4-3200 это около 50 ГБ/с). Сделать вывод о том, насколько эффективно код использует ресурсы памяти. Обсудить, почему операция простого копирования в C++ может быть быстрее или медленнее векторизованной операции в NumPy в зависимости от размера данных и попадания в кэш L3.

---

## Задача 2. Замер времени работы алгоритма вычисления скалярного произведения с разными параметрами компиляции

**Описание задачи из предметной области:** Вычисление спектральных индексов (например,  $NDVI = (NIR - Red) / (NIR + Red)$ ) или классификация по методу наименьших расстояний требует многократного вычисления скалярного произведения векторов спектральной яркости. Оптимизация этой базовой операции напрямую влияет на время обработки всего снимка.

**План решения:** 1. Реализовать функцию вычисления скалярного произведения двух векторов размерности  $N = 10^7$  элементов типа float. 2. Скомпилировать C++ код с различными наборами флагов оптимизации: -O0, -O2, -O3, -O3 -march=native -ffast-math. 3. Замерить время выполнения для каждого варианта компиляции. 4. Реализовать аналогичный замер на Python (с использованием цикла for и векторизованной функции numpy.dot).

**Пошаговое описание решения задачи:** 1. Написать функцию dot\_product на C++, использующую простой цикл for. 2. Создать CMakeLists.txt или скрипт компиляции, позволяющий легко менять флаги. 3. Использовать std::chrono::high\_resolution\_clock для точного замера времени внутри C++ программы. 4. В Python сравнить производительность нативного цикла (как эталон медленной работы) и оптимизированного numpy.dot (который под капотом использует оптимизированные C/Fortran библиотеки, такие как OpenBLAS или MKL). 5. Построить таблицу или график зависимости времени выполнения от уровня оптимизации компилятора.

Программный код на Python:

```
import numpy as np
import time

def benchmark_dot_product_python():
    n = 10_000_000
    a = np.random.rand(n).astype(np.float32)
    b = np.random.rand(n).astype(np.float32)

    print(f"Размерность векторов: {n}")

    # 1. Нативный цикл Python (эталон медленной работы)
    print("Замер нативного цикла Python (может занять время)...")
    start = time.perf_counter()
    res_native = 0.0
    for i in range(n):
        res_native += a[i] * b[i]
    end = time.perf_counter()
    print(f"Нативный цикл Python: {end - start:.4f} сек")

    # 2. Векторизованная операция NumPy
    print("Замер numpy.dot...")
    start = time.perf_counter()
    res_numpy = np.dot(a, b)
    end = time.perf_counter()
    print(f"NumPy dot: {end - start:.6f} сек")

    print(f"Результаты совпадают: {np.isclose(res_native, res_numpy, rtol=1e-4)}")

if __name__ == "__main__":
    benchmark_dot_product_python()
```

Программный код на C++:

```

#include <iostream>
#include <vector>
#include <chrono>
#include <numeric>

// Скалярное произведение
float dot_product(const float* a, const float* b, size_t n) {
    float sum = 0.0f;
    for (size_t i = 0; i < n; ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}

int main() {
    const size_t n = 10_000_000;
    std::vector<float> a(n, 1.5f);
    std::vector<float> b(n, 2.5f);

    std::cout << "Размерность векторов: " << n << "\n";

    // Для демонстрации разных флагов, этот код должен быть скомпилирован
    // несколько раз с разными флагами, например:
    // g++ -O0 main.cpp -o test_00
    // g++ -O3 -march=native -ffast-math main.cpp -o test_03_native

    auto start = std::chrono::high_resolution_clock::now();

    float result = dot_product(a.data(), b.data(), n);

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> elapsed = end - start;

    std::cout << "Результат скалярного произведения: " << result << "\n";
    std::cout << "Время выполнения: " << elapsed.count() << " мс\n";

    return 0;
}

```

**Анализ решения и формулирование выводов:** Проанализировать таблицу результатов. Ожидается, что переход от -O0 к -O3 даст значительное ускорение (в 5-10 раз) за счёт авто-векторизации цикла компилятором в инструкции AVX2/FMA. Добавление -march=native позволит использовать специфичные для процессора инструкции. Сделать вывод о критической важности правильных флагов компиляции в НРС. Сравнить с Python: нативный цикл будет крайне медленным, тогда как numpy.dot покажет результаты, сопоставимые с оптимизированным C++ кодом, что демонстрирует важность использования специализированных библиотек в высокоуровневых языках.

---

### Задача 3. Анализ ассемблерного кода в Compiler Explorer (Godbolt)

**Описание задачи из предметной области:** Для глубокого понимания того, как компилятор преобразует высокоуровневый код в машинные инструкции, необходимо уметь читать ассемблерный код. Это позволяет разработчику распределённых систем убедиться, что компилятор действительно применил векторизацию и развёртку циклов, а не сгенерировал неэффективный скалярный код.

**План решения:** 1. Взять исходный код функции скалярного произведения из Задачи 2. 2. Открыть сайт [godbolt.org](https://godbolt.org). 3. Выбрать компилятор x86-64 gcc (версии 11 или новее). 4. Ввести код и последовательно менять флаги компиляции в поле "Compiler options". 5. Проанализировать изменения в генерируемом ассемблерном коде.

**Пошаговое описание решения задачи:** 1. Скопировать минималистичный C++ код функции скалярного произведения в левое окно Godbolt. 2. Установить флаги -O0. Найти в ассемблерном коде (правое окно) метки цикла, инструкции чтения из памяти (mov), умножения (mulss или mulsd) и сложения. Обратит внимание на обилие инструкций управления стеком и отсутствие векторных регистров. 3. Изменить флаги на -O3 -march=native -ffast-math. 4. Найти в новом коде использование векторных регистров (например, ymm0, zmm0 для AVX2/AVX-512). 5. Идентифицировать инструкции векторного умножения с накоплением (Fused Multiply-Add), такие как vfmadd231ps. 6. Зафиксировать скриншоты или выдержки из Godbolt для отчёта.

Программный код на Python:

*# Данный фрагмент представляет собой заглушку для структуры документа.  
# Основной анализ ассемблерного кода выполняется в браузере на сайте [godbolt.org](https://godbolt.org).  
# Ниже приведён пример скрипта, который мог бы использоваться для автоматической отправки кода в Compiler Explorer API, хотя в рамках лабораторной работы рекомендуется ручная работа в веб-интерфейсе для наглядности.*

```
import requests
import json

def godbolt_api_example():
    url = "https://godbolt.org/api/compiler/g112/compile"
    payload = {
        "source": "float dot(const float* a, const float* b, int n) {\n    float\n    s = 0;\n    for(int i=0; i<n; ++i) s += a[i]*b[i];\n    return s;\n}",
        "options": "-O3 -march=native -ffast-math",
        "compiler": "g112"
    }
    # Примечание: реальный API Godbolt требует специфического формата запроса.
    # Этот код является концептуальным примером.
    print("Для анализа перейдите на https://godbolt.org и вставьте код из фрагмента 6.")

if __name__ == "__main__":
    godbolt_api_example()
```

Программный код на C++:

```
// Код для вставки в Compiler Explorer (godbolt.org)
// Компилятор: x86-64 gcc 11.2 или новее
// Попробуйте менять флаги: -O0, -O3, -O3 -march=native, -O3 -march=native -ffast-math
```

```
float dot_product_basic(const float* a, const float* b, int n) {
    float sum = 0.0f;
    for (int i = 0; i < n; ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}
```

```
// Вариант с __restrict__, помогающий компилятору доказать отсутствие алиасинга
float dot_product_restrict(const float* __restrict__ a, const float* __restrict__
_ b, int n) {
    float sum = 0.0f;
    for (int i = 0; i < n; ++i) {
        sum += a[i] * b[i];
    }
    return sum;
}
```

**Анализ решения и формулирование выводов:** Описать, какие именно ассемблерные инструкции появились при включении оптимизации. Объяснить принцип работы инструкции FMA (выполнение умножения и сложения за один такт с одним округлением, что повышает и скорость, и точность). Сделать вывод о том, что современный компилятор способен полностью векторизовать простой цикл скалярного произведения, устраняя необходимость в ручной написании интринсиков, при условии корректной настройки флагов и отсутствия алиасинга указателей.

## 7. Индивидуальные задания

Студент выполняет задание согласно своему варианту. Вариант определяется последней цифрой зачетной книжки.

### Задание 1

Необходимо реализовать указанную операцию на C++ и Python/NumPy, замерить время выполнения, проанализировать узкие места (memory-bound vs compute-bound, cache misses) и оценить влияние заданного флага компиляции.

| Вар. | Задача<br>(Линейно-<br>алгебраическа<br>я операция) | Тип данных и<br>Размерность | Специфически<br>й флаг<br>компиляции<br>(C++) | Контекст<br>применения в<br>НРС / ML /<br>Физике |
|------|-----------------------------------------------------|-----------------------------|-----------------------------------------------|--------------------------------------------------|
| 1    | <b>АХРУ:</b> $y = \alpha x + y$<br>(векторное       | float32,<br>векторы $10^7$  | -O3 -<br>march=native                         | Базовая<br>операция<br>итерационных              |

| Вар. | Задача<br>(Линейно-<br>алгебраическая операция)                              | Тип данных и<br>Размерность               | Специфический<br>флаг<br>компиляции<br>(C++) | Контекст<br>применения в<br>НРС / ML /<br>Физике                                 |
|------|------------------------------------------------------------------------------|-------------------------------------------|----------------------------------------------|----------------------------------------------------------------------------------|
|      | обновление)                                                                  |                                           |                                              | решателей<br>(Крыловские<br>подпространства).                                    |
| 2    | <b>L2-норма<br/>вектора:</b><br>$\ x\ _2 = \sqrt{\sum x_i^2}$                | float64,<br>вектор $10^7$                 | -O3 -ffast-<br>math                          | Нормализация<br>в машинном<br>обучении,<br>вычисление<br>невязок.                |
| 3    | <b>L1-норма<br/>вектора:</b><br>$\ x\ _1 = \sum  x_i $                       | float32,<br>вектор $10^8$                 | -O3 -<br>march=native                        | Lasso-<br>регуляризация<br>, задачи<br>разреженной<br>оптимизации.               |
| 4    | <b>Косинусное<br/>сходство:</b><br>$\frac{x \cdot y}{\ x\ _2 \ y\ _2}$       | float32,<br>векторы $10^6$                | -O3 -ffast-<br>math -fno-<br>math-errno      | Поиск похожих<br>объектов<br>(ReID),<br>семантически<br>й поиск<br>(Embeddings). |
| 5    | <b>Адамара<br/>произведение</b><br>(поэлементно<br>е): $z_i = x_i \cdot y_i$ | float32,<br>векторы $10^7$                | -O3 -<br>march=native                        | Вейвлет-<br>преобразование,<br>поэлементное<br>масштабирование<br>признаков.     |
| 6    | <b>GEMV:</b><br>Умножение<br>матрицы на<br>вектор $y = Ax$                   | float32,<br>матрица<br>$2000 \times 4000$ | -O3 -<br>march=native                        | Ядро методов<br>понижения<br>размерности,<br>скрытые слои<br>нейросетей.         |
| 7    | <b>Транспонированное<br/>GEMV:</b> $y = A^T x$                               | float64,<br>матрица<br>$3000 \times 3000$ | -O2 -funroll-<br>loops                       | Обратное<br>распространение<br>ошибки<br>(Backpropagation) в ML.                 |
| 8    | <b>Rank-1<br/>обновление:</b><br>$A = A + \alpha xy^T$                       | float32,<br>матрица<br>$2000 \times 2000$ | -O3 -<br>march=native                        | Алгоритм QR-<br>разложения,<br>метод<br>сопряженных<br>градиентов.               |

| Вар. | Задача<br>(Линейно-<br>алгебраическая операция)                                       | Тип данных и<br>Размерность        | Специфический<br>флаг<br>компиляции<br>(C++) | Контекст<br>применения в<br>НРС / ML /<br>Физике                            |
|------|---------------------------------------------------------------------------------------|------------------------------------|----------------------------------------------|-----------------------------------------------------------------------------|
| 9    | <b>Решение<br/>треугольной<br/>системы:</b><br>$Lx = b$<br>(Forward<br>substitution)  | float64,<br>матрица<br>2000 × 2000 | -O3 -ffast-<br>math                          | Прямой и<br>обратный ход<br>в методе<br>Гаусса и LU-<br>разложении.         |
| 10   | <b>Симметрично<br/>е GEMV:</b> $y =$<br>$Ax$ (где A -<br>симметричная<br>)            | float32,<br>матрица<br>3000 × 3000 | -O3 -<br>march=native                        | Вычисление<br>собственных<br>значений<br>(метод<br>Ланцоша).                |
| 11   | <b>GEMM<br/>(Наивное):</b><br>$C = AB$ (без<br>блокировки)                            | float32,<br>матрицы<br>1000 × 1000 | -O0 (для<br>сравнения)                       | Эталон<br>неоптимизиро<br>ванного кода,<br>анализ cache<br>thrashing.       |
| 12   | <b>GEMM<br/>(Оптимизиро<br/>ванное):</b> $C =$<br>$AB$ (с<br>блокировкой/til<br>e)    | float32,<br>матрицы<br>2000 × 2000 | -O3 -<br>march=native<br>-ffast-math         | Ядро<br>сверточных<br>нейросетей и<br>трансформеро<br>в (Attention).        |
| 13   | <b>Транспониро<br/>ванное<br/>GEMM:</b> $C =$<br>$A^T B$                              | float64,<br>матрицы<br>1500 × 1500 | -O3 -<br>march=native                        | Вычисление<br>матриц<br>ковариации,<br>метод главных<br>компонент<br>(PCA). |
| 14   | <b>Сложение<br/>матриц с<br/>масштабиров<br/>анием:</b> $C =$<br>$\alpha A + \beta B$ | float32,<br>матрицы<br>4000 × 4000 | -O3 -<br>march=native                        | Линейные<br>комбинации<br>состояний в<br>динамических<br>системах.          |
| 15   | <b>След<br/>матрицы:</b><br>$Tr(A) = \sum A_{ii}$                                     | float64,<br>матрица<br>5000 × 5000 | -O2 -funroll-<br>loops                       | Вычисление<br>инвариантов,<br>трассировка<br>квантовых<br>операторов.       |
| 16   | <b>Норма<br/>Фробениуса:</b><br>$\ A\ _F = \sqrt{\sum A_{ij}^2}$                      | float32,<br>матрица<br>3000 × 3000 | -O3 -ffast-<br>math                          | Оценка<br>ошибки<br>аппроксимаци<br>и,                                      |

| Вар. | Задача<br>(Линейно-<br>алгебраическая операция)             | Тип данных и<br>Размерность                         | Специфический<br>флаг<br>компиляции<br>(C++) | Контекст<br>применения в<br>НРС / ML /<br>Физике             |
|------|-------------------------------------------------------------|-----------------------------------------------------|----------------------------------------------|--------------------------------------------------------------|
|      |                                                             |                                                     |                                              | регуляризация весов нейросетей.                              |
| 17   | <b>Транспонирование матрицы</b><br>(Out-of-place)           | float32,<br>матрица<br>4000 × 4000                  | -O3 -<br>march=native                        | Классическая задача на оптимизацию паттернов доступа к кэшу. |
| 18   | <b>In-place транспонирование</b><br>(квадратной матрицы)    | float32,<br>матрица<br>3000 × 3000                  | -O3 -funroll-loops                           | Экономия памяти при работе с большими графами и тензорами.   |
| 19   | <b>Блочное транспонирование</b> (Tile-based)                | float32,<br>матрица<br>4000 × 4000,<br>тайл 32 × 32 | -O3 -<br>march=native                        | Оптимизация памяти для SIMD-выровненных архитектур.          |
| 20   | <b>Извлечение диагонали</b> в вектор                        | float64,<br>матрица<br>5000 × 5000                  | -O2 -<br>march=native                        | Инициализация предобуславливателей (preconditioners).        |
| 21   | <b>Метод Якоби</b><br>(1 итерация решения $Ax = b$ )        | float32,<br>матрица<br>2000 × 2000                  | -O3 -ffast-math                              | Параллельные итерационные методы для разреженных СЛАУ.       |
| 22   | <b>Метод Гаусса-Зейделя</b> (1 итерация)                    | float64,<br>матрица<br>2000 × 2000                  | -O3 -<br>march=native                        | Последовательные итерационные методы, теплопередача.         |
| 23   | <b>Степенной метод</b> (1 итерация поиска $\lambda_{max}$ ) | float32,<br>матрица<br>3000 × 3000                  | -O3 -ffast-math                              | Оценка спектрального радиуса, PageRank в                     |

| Вар. | Задача<br>(Линейно-<br>алгебраическая операция)                           | Тип данных и<br>Размерность                            | Специфический<br>флаг<br>компиляции<br>(C++) | Контекст<br>применения в<br>НРС / ML /<br>Физике              |
|------|---------------------------------------------------------------------------|--------------------------------------------------------|----------------------------------------------|---------------------------------------------------------------|
| 24   | <b>Шаг градиентного спуска:</b> $x_{k+1} = x_k - \alpha \nabla f(x_k)$    | float64,<br>вектор $10^7$                              | -O3 -<br>march=native                        | графах.<br>Базовый шаг оптимизации в глубоком обучении.       |
| 25   | <b>Шаг метода сопряженных градиентов (CG)</b>                             | float32,<br>векторы $10^7$                             | -O3 -ffast-math                              | Решение больших симметричных положительно определенных СЛАУ.  |
| 26   | <b>Шаг разложения Холецкого</b><br>(для 1 строки/столбца)                 | float64,<br>матрица $2000 \times 2000$                 | -O3 -<br>march=native                        | Факторизация ковариационных матриц, метод Монте-Карло.        |
| 27   | <b>Ортогонализация Грама-Шмидта</b> (1 вектор)                            | float32,<br>матрица $1000 \times 1000$                 | -O3 -ffast-math                              | Построение ортонормированных базисов, QR-разложение.          |
| 28   | <b>SpMV</b><br>(Умножение разреженной матрицы на вектор, формат CSR)      | float32,<br>матрица $10000 \times 10000$<br>(разр. 1%) | -O3 -<br>march=native                        | Ядро графовых алгоритмов, метод конечных элементов (FEM).     |
| 29   | <b>Softmax</b><br>(векторизованный): $z_i = \frac{e^{x_i}}{\sum e^{x_j}}$ | float32,<br>вектор $10^6$                              | -O3 -ffast-math -fno-math-errno              | Финальный слой классификационных нейросетей (Attention).      |
| 30   | <b>RMSNorm</b><br>(Root Mean Square Layer Normalization)                  | float32,<br>вектор $10^5$ ,<br>батч 64                 | -O3 -<br>march=native -ffast-math            | Нормализация в современных LLM (например, архитектура LLaMA). |

## Методические указания к выполнению индивидуального задания:

1. **Анализ вычислительной сложности:** Для выбранной операции определите теоретическую сложность (количество операций FLOPs) и объем перемещаемых данных (в байтах). Рассчитайте *Arithmetic Intensity* (FLOPs / Byte).
2. **Классификация задачи:** На основе *Arithmetic Intensity* определите, является ли задача *memory-bound* (ограничена памятью, например, операции Level 1 BLAS) или *compute-bound* (ограничена вычислениями, например, GEMM).
3. **Реализация и замер:**
  - Реализуйте операцию на C++. Убедитесь, что массивы выровнены (например, с помощью `std::aligned_alloc` или `alignas(64)`), чтобы компилятор мог безопасно использовать SIMD.
  - Используйте `std::chrono::high_resolution_clock` для замера времени. Усредните результат по 10-50 запускам.
4. **Анализ ассемблера:** Откройте сгенерированный код в Godbolt. Проверьте, применил ли компилятор векторизацию (инструкции `ymm/zmm`) и FMA. Если векторизации нет — проанализируйте причину (алиасинг, сложные ветвления, нестрогие стандарты IEEE 754).
5. **Влияние флага:** Сравните время выполнения с флагом из вашего варианта и с базовым флагом `-O0`. Объясните, почему заданный специфический флаг (например, `-ffast-math` или `-funroll-loops`) дал именно такой прирост производительности для данной конкретной математической операции.

## Задание 2

### 1. Общая постановка задачи

Целью индивидуального задания является разработка **стохастической (вероятностной) математической модели** распределённой вычислительной системы (РВС). В отличие от детерминированных моделей, которые оперируют средними значениями, предлагаемая модель должна учитывать **статистические характеристики** (математическое ожидание, дисперсию, квантили) времени выполнения базовых алгоритмов линейной алгебры, полученные в ходе лабораторных замеров, а также **аппаратные ограничения** конкретного вычислительного оборудования.

Студенту необходимо: 1. Провести серию бенчмарков выбранного алгоритма линейной алгебры на целевом оборудовании. 2. Собрать статистические данные (выборка из  $N \geq 100$  запусков). 3. Разработать математическую модель, предсказывающую время выполнения или пропускную способность распределённой системы с учётом выявленных статистических флуктуаций и аппаратных «узких мест». 4. Верифицировать модель, сравнив предсказанные значения с реальными замерами на увеличенном объёме данных или большем числе узлов.

---

## 2. Индивидуальные задания (30 вариантов)

Студент выполняет задание согласно своему варианту. Вариант определяется преподавателем или последней цифрой зачетной книжки.

| Вар. | Архитектура вычислительной системы  | Базовый алгоритм (Линейная алгебра)             | Математический аппарат модели                       | Учитываемые аппаратные характеристики и статистические данные              |
|------|-------------------------------------|-------------------------------------------------|-----------------------------------------------------|----------------------------------------------------------------------------|
| 1    | Гомогенный CPU-кластер (MPI)        | Плотное матричное умножение (GEMM)              | Расширенная модель Roofline с учётом коммуникаций   | Пропускная способность InfiniBand HDR, статистика хвостовых задержек (P99) |
| 2    | Гомогенный CPU-кластер (MPI)        | Разреженное матрично-векторное умножение (SpMV) | Теория массового обслуживания (СМО M/G/c)           | Задержка сети RoCEv2, дисперсия времени вычислений из-за cache-miss        |
| 3    | Одноузловая NUMA-система (2 сокета) | Плотное GEMM ( $C = AB$ )                       | Модель асимметричного доступа к памяти (NUMA-aware) | Статистика межсокетных переходов, латентность DDR4 vs локальной памяти     |
| 4    | Гетерогенный узел (CPU + GPU)       | Сингулярное разложение (SVD)                    | Модель Amdahl с вероятностной оценкой ускорения     | Статистика задержек PCIe 4.0 x16, дисперсия времени передачи данных        |
| 5    | Гетерогенный узел (2x GPU)          | Тензорное свёрточное умножение                  | Модель Gustafson для гетерогенных сред              | Пропускная способность NVLink, статистика синхронизации и потоков          |
| 6    | Гомогенный                          | Разложение                                      | Стохастическая                                      | Статистика                                                                 |

| Вар. | Архитектура<br>вычислительн<br>ой системы        | Базовый<br>алгоритм<br>(Линейная<br>алгебра)           | Математическ<br>ий аппарат<br>модели                                  | Учитываемые<br>аппаратные<br>характеристик<br>и и<br>статистически<br>е данные            |
|------|--------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
|      | CPU-кластер<br>(MPI)                             | Холецкого                                              | я модель<br>коммуникацио<br>нно-<br>вычислительн<br>ого<br>перекрытия | времени<br>ожидания<br>MPI_Wait,<br>пропускная<br>способность<br>сети                     |
| 7    | Распределённ<br>ая файловая<br>система<br>(HDFS) | Быстрое<br>преобразован<br>ие Фурье<br>(FFT)           | Модель<br>очереди с<br>приоритетами<br>(M/M/1 с<br>прерываниями<br>)  | Дисперсия<br>времени<br>чтения блоков<br>с NVMe SSD,<br>сетевой<br>джиттер                |
| 8    | Гомогенный<br>CPU-кластер<br>(MPI)               | Метод<br>сопряженных<br>градиентов<br>(CG)             | Марковские<br>модели<br>вознагражден<br>ия (MRM)                      | Статистика<br>сходимости<br>метода,<br>влияние<br>сетевых<br>задержек на<br>итерации      |
| 9    | Одноузловая<br>NUMA-<br>система (4<br>сокета)    | Ортогонализа<br>ция Грама-<br>Шмидта                   | Модель кэш-<br>промахов<br>(Miss Rate<br>Model)                       | Статистика L3<br>cache<br>thrashing,<br>пропускная<br>способность<br>QPI/UPI              |
| 10   | Гетерогенный<br>кластер (CPU<br>+ FPGA)          | Фильтр<br>Калмана<br>(матричные<br>операции)           | Модель<br>надёжности<br>систем с<br>резервирован<br>ием               | Статистика<br>задержек<br>PCIe,<br>дисперсия<br>времени<br>выполнения<br>на FPGA          |
| 11   | Гомогенный<br>CPU-кластер<br>(MPI)               | LU-<br>разложение с<br>выбором<br>главного<br>элемента | Модель<br>балансировки<br>нагрузки (Min-<br>Min / Max-Min)            | Статистика<br>дисбаланса<br>времени<br>выполнения<br>процессов,<br>сетевая<br>латентность |
| 12   | Кластер с<br>общей                               | Вычисление<br>собственных                              | Модель<br>синхронизаци                                                | Дисперсия<br>времени                                                                      |

| Вар.      | Архитектура<br>вычислительной<br>системы      | Базовый<br>алгоритм<br>(Линейная<br>алгебра)          | Математический аппарат<br>модели                                  | Учитываемые<br>аппаратные<br>характеристик<br>и и<br>статистически<br>е данные         |
|-----------|-----------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|           | памятью<br>(PGAS/UPC)                         | значений (QR)                                         | и потоков<br>(Petri nets)                                         | атомарных<br>операций,<br>латентность<br>межъядерных<br>шин                            |
| <b>13</b> | Гомогенный<br>CPU-кластер<br>(MPI)            | Обратная<br>матрица<br>(Gauss-Jordan)                 | Модель<br>отказоустойчи<br>вости (Markov<br>availability)         | Статистика<br>сбоев узлов<br>(MTBF), время<br>контрольных<br>точек<br>(checkpointing)  |
| <b>14</b> | Гетерогенный<br>узел (CPU +<br>GPU)           | Решение<br>систем<br>линейных<br>уравнений<br>(GMRES) | Модель<br>энергопотребл<br>ения и<br>производител<br>ьности (EDP) | Статистика<br>тепловых<br>дросселей<br>(throttling),<br>дисперсия<br>частоты GPU       |
| <b>15</b> | Одноузловая<br>NUMA-<br>система (2<br>сокета) | Кронекеро<br>произведе<br>ние матриц                  | Модель<br>локальности<br>данных (Data<br>Locality Model)          | Статистика<br>TLB-miss,<br>влияние<br>размера Huge<br>Pages на<br>дисперсию<br>времени |
| <b>16</b> | Гомогенный<br>CPU-кластер<br>(MPI)            | Вычисление<br>определителя<br>матрицы                 | Модель<br>масштабируе<br>мости (Sunni &<br>Ni model)              | Статистика<br>накладных<br>расходов на<br>сериализацию<br>/десериализац<br>ию MPI      |
| <b>17</b> | Гетерогенный<br>узел (CPU +<br>NPU)           | Умножение<br>тензоров<br>(TPU-<br>операции)           | Модель<br>квантования и<br>точности<br>вычислений                 | Статистика<br>ошибок<br>округления<br>INT8/FP16,<br>дисперсия<br>времени<br>инференса  |
| <b>18</b> | Гомогенный<br>CPU-кластер<br>(MPI)            | Разреженное<br>матричное<br>умножение<br>(SpGEMM)     | Модель<br>динамическог<br>о<br>распределени                       | Статистика<br>фрагментации<br>памяти,<br>дисперсия                                     |

| Вар. | Архитектура<br>вычислительной<br>системы      | Базовый<br>алгоритм<br>(Линейная<br>алгебра)          | Математический аппарат<br>модели                                         | Учитываемые<br>аппаратные<br>характеристик<br>и и<br>статистически<br>е данные             |
|------|-----------------------------------------------|-------------------------------------------------------|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
|      |                                               |                                                       | я памяти                                                                 | времени<br>аллокации<br>(malloc)                                                           |
| 19   | Одноузловая<br>NUMA-<br>система (2<br>сокета) | Тридиагональ<br>ный решатель<br>(Thomas<br>algorithm) | Модель<br>конвейерной<br>обработки<br>(Pipeline<br>model)                | Статистика<br>простоев<br>конвейера из-<br>за ветвлений,<br>латентность<br>памяти          |
| 20   | Гетерогенный<br>узел (CPU +<br>GPU)           | Быстрое<br>преобразование<br>Хаара<br>(вейвлеты)      | Модель<br>асинхронных<br>вычислений<br>(Bulk<br>Synchronous<br>Parallel) | Статистика<br>времени<br>синхронизаци<br>и барьеров,<br>дисперсия<br>PCIe transfers        |
| 21   | Гомогенный<br>CPU-кластер<br>(MPI)            | Метод Якоби<br>для<br>разреженных<br>СЛАУ             | Модель<br>диффузии<br>данных в сети                                      | Статистика<br>сетевых<br>коллизий,<br>пропускная<br>способность<br>коммутаторов            |
| 22   | Кластер с<br>общей<br>памятью<br>(OpenMP)     | Псевдообраще<br>ние Мура-<br>Пенроуза                 | Модель<br>вероятностног<br>о<br>кэширования<br>(Probabilistic<br>Cache)  | Статистика<br>попаданий в<br>L2/L3 кэш,<br>дисперсия<br>времени<br>доступа к RAM           |
| 23   | Гомогенный<br>CPU-кластер<br>(MPI)            | Вычисление<br>матричной<br>экспоненты                 | Модель<br>стохастическо<br>го<br>градиентного<br>спуска                  | Статистика<br>флуктуаций<br>градиентов,<br>влияние<br>сетевых<br>задержек на<br>сходимость |
| 24   | Гетерогенный<br>узел (CPU +<br>GPU)           | Разложение<br>по<br>собственным<br>векторам           | Модель<br>теплоотвода и<br>производитель<br>ности                        | Статистика<br>температурны<br>х режимов,<br>дисперсия<br>тактовой<br>частоты<br>(boost)    |

| Вар. | Архитектура<br>вычислительн<br>ой системы        | Базовый<br>алгоритм<br>(Линейная<br>алгебра)  | Математическ<br>ий аппарат<br>модели                              | Учитываемые<br>аппаратные<br>характеристик<br>и и<br>статистически<br>е данные            |
|------|--------------------------------------------------|-----------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 25   | Одноузловая<br>NUMA-<br>система (4<br>сокета)    | Умножение<br>блочных<br>матриц                | Модель<br>оптимизации<br>тайлинга (Tile<br>Size<br>Optimization)  | Статистика<br>влияния<br>размера тайла<br>на cache-<br>misses,<br>дисперсия<br>времени    |
| 26   | Гомогенный<br>CPU-кластер<br>(MPI)               | Метод Гаусса-<br>Зейделя                      | Модель<br>асинхронных<br>итераций<br>(Asynchronous<br>Iterations) | Статистика<br>устаревания<br>данных<br>(staleness),<br>дисперсия<br>сетевых<br>задержек   |
| 27   | Гетерогенный<br>узел (CPU +<br>GPU)              | Свертка<br>(Conv2D) в<br>линейной<br>алгебре  | Модель<br>имбаланса<br>вычислительн<br>ых ядер                    | Статистика<br>загрузки SM<br>(Streaming<br>Multiprocessor<br>s), дисперсия<br>warp-stalls |
| 28   | Гомогенный<br>CPU-кластер<br>(MPI)               | QR-<br>разложение<br>(метод<br>Хаусхолдера)   | Модель<br>сетевых<br>широковещате<br>льных<br>операций            | Статистика<br>времени<br>MPI_Bcast,<br>влияние<br>топологии<br>дерева на<br>дисперсию     |
| 29   | Одноузловая<br>NUMA-<br>система (2<br>сокета)    | Вычисление<br>нормы<br>матрицы<br>(Frobenius) | Модель<br>энергоэффект<br>ивности<br>(Performance<br>per Watt)    | Статистика<br>скачков<br>энергопотребл<br>ения,<br>дисперсия<br>напряжения<br>(Vcore)     |
| 30   | Гетерогенный<br>кластер (CPU<br>+ GPU +<br>FPGA) | Комплексная<br>тензорная<br>сеть (MPS)        | Мультиагентн<br>ая модель<br>распределени<br>я задач              | Статистика<br>гетерогенных<br>задержек,<br>дисперсия<br>времени<br>отклика<br>ускорителей |

---

### 3. План выполнения задания

#### Этап 1: Сбор статистических данных (Бенчмаркинг)

1. Реализовать базовый алгоритм (согласно варианту) на C++ (с использованием OpenMP/MPI/CUDA в зависимости от архитектуры).
2. Провести  $N = 100$  (или более) запусков алгоритма на фиксированном объеме данных.
3. Для каждого запуска зафиксировать:
  - $t_i$  — время выполнения (через `std::chrono::high_resolution_clock`).
  - Аппаратные метрики (через `perf` или `nvprof`): количество cache-misses, объем переданных данных по сети/шине.
4. Рассчитать выборочные характеристики:
  - Математическое ожидание (среднее):  $\bar{t} = \frac{1}{N} \sum_{i=1}^N t_i$
  - Дисперсию:  $D(t) = \frac{1}{N-1} \sum_{i=1}^N (t_i - \bar{t})^2$
  - Стандартное отклонение:  $\sigma(t) = \sqrt{D(t)}$
  - Квантили (P50, P95, P99) для оценки хвостовых задержек.

#### Этап 2: Разработка математической модели

На основе выбранного математического аппарата (см. таблицу вариантов) сформулировать уравнения модели.

*Пример для Варианта 1 (Расширенная модель Roofline):* Время выполнения распределённой задачи  $T_{total}$  моделируется как:

$$T_{total} = \max\left(\frac{F}{P_{peak}}, \frac{M}{BW_{mem}}\right) + T_{comm} + T_{sync}$$

где: \*  $F$  — количество операций (FLOPs),  $P_{peak}$  — пиковая производительность CPU. \*  $M$  — объем перемещаемых данных,  $BW_{mem}$  — пропускная способность памяти. \*

$T_{comm} = \frac{D_{net}}{BW_{net}} + L_{net}$  — время коммуникации, где  $D_{net}$  — объем данных,  $BW_{net}$  — пропускная способность сети,  $L_{net}$  — латентность. \* **Стохастическая часть:**  $BW_{net}$  и  $L_{net}$  заменяются на случайные величины, подчиняющиеся распределению, полученному из статистики бенчмарков (например, нормальное или логнормальное для хвостовых задержек). Модель должна вычислять не просто  $T_{total}$ , а функцию распределения  $F(T_{total})$  или доверительный интервал.

#### Этап 3: Параметризация и верификация

1. Подставить реальные технические характеристики оборудования (из спецификаций CPU, RAM, сети) и полученные статистические данные в модель.
2. Рассчитать предсказанное моделью время выполнения для *нового* объема данных (например, в 2 или 4 раза больше) или для *увеличенного* числа вычислительных узлов.

3. Провести контрольные замеры на новом объеме/конфигурации.
4. Сравнить предсказание модели с реальностью. Рассчитать относительную ошибку модели:

$$\varepsilon = \frac{|T_{model} - T_{real}|}{T_{real}} \times 100\%$$

---

## 4. Требования к отчёту

Отчёт по индивидуальному заданию должен быть оформлен в виде PDF-документа и содержать следующие разделы:

1. **Титульный лист** (согласно стандартам университета).
  2. **Введение:** Постановка задачи, описание выбранной архитектуры и базового алгоритма.
  3. **Аппаратная конфигурация:** Таблица с точными техническими характеристиками используемого оборудования (модель CPU, частоты, размеры кэшей, тип и частота RAM, модель сетевого адаптера).
  4. **Статистический анализ бенчмарков:**
    - Гистограмма распределения времени выполнения.
    - Таблица статистических метрик (среднее, дисперсия, P95, P99).
    - Анализ причин флуктуаций (влияние ОС, кэш-промахи, сетевой джиттер).
  5. **Математическая модель:**
    - Вывод основных уравнений.
    - Описание того, как статистические данные и аппаратные ограничения интегрированы в модель.
    - Блок-схема алгоритма работы модели.
  6. **Верификация модели:**
    - Сравнение предсказанных и реальных значений.
    - График зависимости ошибки модели от масштаба задачи.
  7. **Заключение:** Выводы о применимости выбранного математического аппарата, анализ «узких мест» системы, рекомендации по оптимизации.
  8. **Приложение:** Листинги кода бенчмарков и скриптов для расчёта модели (Python/C++).
- 

## 5. Критерии оценивания

| Критерий                                  | Максимальный балл | Описание                                                                                    |
|-------------------------------------------|-------------------|---------------------------------------------------------------------------------------------|
| <b>Качество бенчмаркинга и статистики</b> | 20                | Корректность сбора данных, обоснованность размера выборки, глубина статистического анализа. |
| <b>Строгость математической модели</b>    | 40                | Обоснованность выбора математического аппарата, корректность                                |

| Критерий                                | Максимальный балл | Описание                                                                                                                                                  |
|-----------------------------------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Верификация и анализ результатов</b> | 20                | формул, учёт стохастической природы и аппаратных ограничений.<br>Качество сравнения модели с реальностью, анализ причин расхождений, построение графиков. |
| <b>Качество оформления отчёта</b>       | 10                | Научный стиль, структура, наличие схем и графиков, аккуратность листингов.                                                                                |
| <b>Защита задания</b>                   | 10                | Глубина понимания материала, способность ответить на вопросы преподавателя по модели и архитектуре.                                                       |
| <b>ИТОГО</b>                            | <b>100</b>        |                                                                                                                                                           |

## 8. Тестовые задания с ответами по теме работы

### Часть 1. Выбор одного варианта из нескольких (5 вопросов)

**Вопрос 1.** Какова теоретическая пропускная способность двухканальной памяти DDR4-3200? а) 12.8 ГБ/с

б) 25.6 ГБ/с

в) 51.2 ГБ/с

г) 102.4 ГБ/с

*Правильный ответ: в. (3200 млн транзакций/с 8 байт \* 2 канала = 51200 МБ/с ≈ 51.2 ГБ/с).\**

**Вопрос 2.** Какая инструкция процессора позволяет выполнить умножение и сложение за один такт, повышая как производительность, так и точность вычислений с плавающей запятой? а) MOV

б) ADD

в) FMA (Fused Multiply-Add)

г) JMP

*Правильный ответ: в.*

**Вопрос 3.** Какой флаг компилятора GCC указывает на генерацию кода, оптимизированного исключительно под архитектуру процессора, на котором происходит компиляция? а) -O3

б) -march=native

в) -Wall

г) -std=c++17

*Правильный ответ: б.*

**Вопрос 4.** Что такое «Стена памяти» (Memory Wall) в контексте высокопроизводительных вычислений? а) Физическое ограничение на размер оперативной памяти в сервере.

б) Разрыв в скорости роста производительности процессоров и скорости доступа к памяти, из-за чего процессор простаивает в ожидании данных.

в) Защита оперативной памяти от несанкционированного доступа.

г) Ограничение на количество потоков, которые может создать процесс.

*Правильный ответ: б.*

**Вопрос 5.** Какая сетевая технология HPC обеспечивает наименьшую задержку (latency) и поддерживает прямой доступ к памяти удалённого узла (RDMA)? а) Gigabit Ethernet

б) Wi-Fi 6

в) InfiniBand

г) Bluetooth

*Правильный ответ: в.*

## Часть 2. Множественный выбор (5 вопросов)

**Вопрос 6.** Какие из перечисленных флагов GCC способствуют повышению производительности за счёт векторизации и агрессивных преобразований? а) -O0

б) -O3

в) -ffast-math

г) -g

*Правильные ответы: б, в.*

**Вопрос 7.** Какие компоненты входят в иерархию памяти современного процессора? а) Регистры

б) Кэш L1, L2, L3

в) Жёсткий диск (HDD)

г) Оперативная память (RAM)

*Правильные ответы: а, б, г.*

**Вопрос 8.** Почему нативный цикл for в Python для вычисления скалярного произведения работает значительно медленнее, чем numpy.dot? а) Python является интерпретируемым языком, и каждая итерация цикла несёт накладные расходы интерпретатора.

б) numpy.dot написан на оптимизированном C/Fortran и использует SIMD-инструкции.

в) Python не поддерживает математические операции.

г) numpy автоматически распараллеливает вычисления на GPU.

*Правильные ответы: а, б.*

**Вопрос 9.** Какие инструменты можно использовать для профилирования производительности в Linux? а) perf

б) Valgrind (Callgrind)

в) Microsoft Word

г) GDB (в режиме профилирования)

*Правильные ответы: а, б, г.*

**Вопрос 10.** Какие факторы влияют на реальную пропускную способность памяти при копировании массива? а) Размер массива (попадание в кэш L3 или выход в RAM).  
б) Выравнивание данных в памяти.  
в) Цвет монитора разработчика.  
г) Паттерн доступа к памяти (последовательный или случайный).

*Правильные ответы: а, б, г.*

### Часть 3. Сопоставление (5 вопросов)

**Вопрос 11.** Установите соответствие между флагом компилятора и его действием: 1)

-O0

2) -O3

3) -march=native

4) -funroll-loops

5) -ffast-math

а) Генерация кода под конкретный CPU.

б) Отключение оптимизаций для отладки.

в) Нарушение строгих стандартов IEEE 754 ради скорости.

г) Агрессивная оптимизация, включая авто-векторизацию.

д) Увеличение размера кода за счёт повторения тела цикла для уменьшения накладных расходов на ветвление.

*Правильные ответы: 1-б, 2-г, 3-а, 4-д, 5-в.*

**Вопрос 12.** Установите соответствие между термином и определением: 1) SIMD

2) NUMA

3) RDMA

4) Cache line

5) FLOPS

а) Архитектура с неоднородным доступом к памяти.

б) Блок данных (обычно 64 байта), переносимый между RAM и кэшем.

в) Мера вычислительной производительности (операций с плавающей запятой в секунду).

г) Выполнение одной инструкции над множеством данных.

д) Прямой доступ к памяти удалённого узла без участия его CPU.

*Правильные ответы: 1-г, 2-а, 3-д, 4-б, 5-в.*

**Вопрос 13.** Установите соответствие между языком/инструментом и его ролью в лабораторной работе: 1) Python time.perf\_counter

2) C++ std::chrono

3) Godbolt

4) GCC

5) NumPy

а) Высокоуровневая библиотека для векторизованных вычислений.

б) Онлайн-инструмент для визуализации ассемблерного кода.

в) Компилятор, преобразующий исходный код в машинный.

г) Средство замера времени с высоким разрешением в Python.

д) Стандартная библиотека C++ для работы с временем.

*Правильные ответы: 1-г, 2-д, 3-б, 4-в, 5-а.*

**Вопрос 14.** Установите соответствие между типом задачи и её характеристикой: 1) Memory-bound  
2) Compute-bound  
3) I/O-bound

- а) Производительность ограничена скоростью вычислений процессора (например, перемножение больших плотных матриц).
- б) Производительность ограничена скоростью чтения/записи в память (например, копирование массива).
- в) Производительность ограничена скоростью дисковой подсистемы или сети.

*Правильные ответы: 1-б, 2-а, 3-в.*

**Вопрос 15.** Установите соответствие между уровнем кэша и его типичными характеристиками: 1) L1  
2) L2  
3) L3

- а) Самый быстрый, наименьший объём (32-64 КБ), свой для каждого ядра.
- б) Самый медленный из кэшей, наибольший объём (несколько МБ), разделяется между ядрами.
- в) Промежуточный уровень, объём 256 КБ - 1 МБ, свой для каждого ядра.

*Правильные ответы: 1-а, 2-в, 3-б.*

#### Часть 4. Ввод пропущенного слова (5 вопросов)

**Вопрос 16.** Архитектура многопроцессорной системы, в которой время доступа к памяти зависит от её расположения относительно процессора, называется \_\_\_\_\_.

*Правильный ответ: NUMA (Non-Uniform Memory Access).*

**Вопрос 17.** Инструкция процессора, выполняющая операцию  $a = b \times c + d$  за один такт, называется \_\_\_\_\_.

*Правильный ответ: FMA (Fused Multiply-Add).*

**Вопрос 18.** Для замера времени выполнения кода в Python с целью получения наиболее точных и монотонных результатов рекомендуется использовать функцию \_\_\_\_\_ из модуля time.

*Правильный ответ: perf\_counter.*

**Вопрос 19.** Процесс автоматического преобразования скалярного цикла компилятором в код, использующий векторные регистры (SIMD), называется авто-\_\_\_\_\_.

*Правильный ответ: векторизацией (или vectorization).*

**Вопрос 20.** Сетевая технология, позволяющая сетевой карте записывать данные напрямую в оперативную память сервера, минуя процессор, поддерживает протокол \_\_\_\_\_.

*Правильный ответ: RDMA (Remote Direct Memory Access).*

## Часть 5. Анализ фрагмента программного кода (5 вопросов)

**Вопрос 21.** Проанализируйте фрагмент кода на C++:

```
float dot_product(const float* a, const float* b, int n) {  
    float sum = 0.0f;  
    for (int i = 0; i < n; ++i) {  
        sum += a[i] * b[i];  
    }  
    return sum;  
}
```

Какую оптимизацию применит компилятор GCC при сборке с флагами `-O3 -march=native -ffast-math`? а) Ничего не изменится, код останется скалярным. б) Цикл будет векторизован с использованием инструкций AVX и FMA для одновременного умножения и сложения нескольких пар элементов. в) Цикл будет полностью удалён как мёртвый код. г) Компилятор выдаст ошибку, так как указатели не проверены на nullptr.  
*Правильный ответ: б.*

**Вопрос 22.** Что произойдёт при выполнении следующего кода на Python?

```
import numpy as np  
import time  
a = np.random.rand(10**7)  
b = np.random.rand(10**7)  
start = time.time()  
res = np.dot(a, b)  
end = time.time()
```

а) Код выполнится очень медленно из-за интерпретируемой природы цикла внутри `np.dot`. б) Код выполнится быстро, так как `np.dot` делегирует вычисление оптимизированным низкоуровневым библиотекам (OpenBLAS/MKL), использующим SIMD и многопоточность. в) Произойдёт переполнение памяти. г) Функция `time.time()` не может измерять такие короткие интервалы.  
*Правильный ответ: б.*

**Вопрос 23.** В ассемблерном коде, сгенерированном Godbolt, вы видите инструкцию `vmovups ymm0, [rdi]`. Что она означает?

а) Перемещение 256 бит (8 значений float) из памяти по адресу `rdi` в векторный регистр `ymm0` без требования выравнивания. б) Сложение двух векторов. в) Переход по адресу `rdi`. г) Сохранение 32-битного значения в стек.  
*Правильный ответ: а.*

**Вопрос 24.** Какой недостаток имеет использование флага `-ffast-math` при обработке научных данных?

- а) Он значительно замедляет выполнение программы.
  - б) Он может изменить численный результат вычислений из-за нарушения ассоциативности операций с плавающей запятой и игнорирования специальных значений (NaN, Inf).
  - в) Он несовместим с языком C++.
  - г) Он отключает все оптимизации.
- Правильный ответ: б.*

**Вопрос 25.** Проанализируйте код замера времени на C++:

```
auto start = std::chrono::system_clock::now();  
// ... вычисления ...  
auto end = std::chrono::system_clock::now();
```

- В чём потенциальная проблема использования `system_clock` для профилирования? а) Он имеет слишком высокое разрешение.
- б) Он может быть изменён системными процессами (например, синхронизацией NTP), что приведёт к отрицательному или некорректному интервалу времени. Для профилирования следует использовать `steady_clock` или `high_resolution_clock`.
- в) Он не поддерживает C++11.
- г) Он измеряет только время работы процессора, а не реальное время.
- Правильный ответ: б.*

## Часть 6. Поиск ошибок в фрагменте программного кода (5 вопросов)

**Вопрос 26.** Найдите ошибку в коде замера пропускной способности памяти на C++:

```
int n = 1000000;  
float* a = new float[n];  
float* b = new float[n];  
// ... инициализация ...  
auto start = std::chrono::high_resolution_clock::now();  
for(int i=0; i<n; ++i) b[i] = a[i];  
auto end = std::chrono::high_resolution_clock::now();  
double time_sec = std::chrono::duration<double>(end - start).count();  
double bw = (n * sizeof(float)) / time_sec; // Ошибка здесь
```

- а) Неправильный тип переменной `time_sec`.
- б) В формуле расчёта пропускной способности (`bw`) учтён только объём чтения (или записи), а не суммарный трафик (чтение + запись). Должно быть  $(2 * n * \text{sizeof(float)}) / \text{time\_sec}$ .
- в) Использование `new` вместо `malloc`.
- г) Цикл `for` не может использоваться для копирования.
- Правильный ответ: б.*

**Вопрос 27.** Найдите ошибку в коде на Python:

```
import time  
a = [0.0] * 10000000  
b = [0.0] * 10000000  
start = time.time()  
for i in range(10000000):
```

```
a[i] = b[i] * 2.0
end = time.time()
```

- а) Использование стандартных списков Python (list) вместо numpy.ndarray для численных расчётов приводит к колоссальным накладным расходам и некорректному сравнению производительности с C++.
- б) Операция \* 2.0 недопустима для списков.
- в) Переменная i не определена.
- г) Функция time.time() не существует.

*Правильный ответ: а.*

**Вопрос 28.** Найдите ошибку в аргументах компиляции: Студент хочет скомпилировать код с максимальной оптимизацией под свой процессор, но пишет: `g++ -O0 -march=native main.cpp`.

- а) Флаг -O0 полностью отключает оптимизации, делая использование -march=native бессмысленным для повышения производительности. Следует использовать -O3.
- б) Флаг -march=native не существует.
- в) Порядок флагов не имеет значения, но -O0 должен быть в конце.
- г) Нельзя использовать -march=native с main.cpp.

*Правильный ответ: а.*

**Вопрос 29.** Найдите ошибку в описании задачи для Godbolt: Студент вставляет в Godbolt следующий код и жалуется, что не видит векторизации:

```
float dot(float* a, float* b, int n) {
    float s = 0;
    for(int i=0; i<n; ++i) s += a[i]*b[i];
    return s;
}
```

Компилятор: x86-64 gcc 11.2, флаги: -O3.

- а) Компилятор не может векторизовать цикл, так как не может доказать отсутствие алиасинга (перекрытия) указателей a и b. Необходимо добавить ключевое слово `__restrict__` или флаг -fno-strict-aliasing / -ffast-math.
- б) В Godbolt нельзя использовать указатели.
- в) Функция должна называться main.
- г) Тип float не поддерживается векторизацией.

*Правильный ответ: а.*

**Вопрос 30.** Найдите логическую ошибку в расчёте размера данных: Студент измеряет пропускную способность при обработке изображения размером 4000x4000 пикселей, тип данных uint16\_t (2 байта), 3 канала. Он считает объём данных как 4000 × 4000 × 2 байта.

- а) Не учтено количество каналов (3). Правильный объём одного кадра: 4000 × 4000 × 3 × 2 байт.
- б) Тип uint16\_t занимает 4 байта.
- в) Разрешение 4000x4000 слишком мало для замера.
- г) Ошибки нет.

*Правильный ответ: а.*

---

## 9. Контрольные вопросы

1. Опишите иерархию памяти современного серверного процессора. Как размер кэш-линии влияет на эффективность обработки растровых данных?
2. В чём заключается фундаментальное различие между архитектурами UMA и NUMA? Как это различие влияет на разработку распределённых систем обработки спутниковых изображений?
3. Дайте определение понятию «пропускная способность памяти» (Memory Bandwidth). Как её рассчитать теоретически и как измерить практически?
4. Объясните принцип работы инструкций SIMD (SSE, AVX). Какую роль играет инструкция FMA (Fused Multiply-Add) в ускорении скалярного произведения?
5. Подробно опишите действие флагов компиляции GCC: `-O3`, `-march=native`, `-ffast-math`, `-funroll-loops`. Какие риски несёт использование `-ffast-math` в научных вычислениях?
6. Почему интерпретируемые языки (Python) проигрывают в производительности компилируемым (C++) в задачах с интенсивными циклами, и как библиотека NumPy нивелирует этот недостаток?
7. Что такое RDMA и почему технологии вроде InfiniBand критически важны для кластерных вычислений, в отличие от стандартного Gigabit Ethernet?
8. Как инструмент Compiler Explorer (Godbolt) помогает разработчику в задачах низкоуровневой оптимизации? Какие ассемблерные инструкции свидетельствуют об успешной векторизации?
9. Что такое «алиасинг указателей» (pointer aliasing) и почему он препятствует авто-векторизации циклов компилятором? Как это обойти?
10. Опишите полный пайплайн оптимизации вычислительного ядра: от написания скалярного кода до анализа ассемблерного вывода и выбора флагов компиляции.